

Appendix

Table of Contents

A	Implementation details	2
A.1	Attention-based demonstration encoding	2
A.2	FiLM-based demonstration encoding	2
A.3	Modification of our method on top of ArticBot [2]	3
B	More simulation results	3
C	Visualization of Real World Results	4

A Implementation details

Here we provide more implementation details on the two baselines we compared to in the simulation experiments: Attention-based demonstration encoding and FiLM-based demonstration encoding.

A.1 Attention-based demonstration encoding

An illustration of this baseline is shown in Figure 1. In this baseline, to process the demonstration information, we first concatenate the demonstration object point cloud o_{demo} with the demonstration grasping end-effector points $\{ee_i^{demo-grasp}\}_{i=1}^4$ to obtain o_{demo}^{grasp} , and also concatenate the demonstration object point cloud o_{demo} with the demonstration opening end-effector points $\{ee_i^{demo-open}\}_{i=1}^4$ to obtain o_{demo}^{open} .

We use a small PointNet++ [1] to encode both o_{demo}^{grasp} and o_{demo}^{open} . The PointNet++ has 4 set abstraction layers. Given an input point cloud, this PointNet++ will generate features (or tokens) of size 64×512 and 16×512 after the 3rd and the 4th set abstraction layer. The feature of o_{demo}^{grasp} and o_{demo}^{open} after the 3rd and 4th set abstraction layers are concatenated together to form the demonstration features f_1^{demo} and f_2^{demo} , which represents the demonstration’s information at two different granularity.

As in Articubot [2], we use another PointNet++ (blue layers in Figure 1) to encode the current observation o to generate the weights and displacements that are used to calculate the final predicted sub-goal end-effector points. This PointNet++ has 6 set abstraction layers, and 6 feature propagation layers. After the 5th and 6th set abstraction layers, the point cloud is encoded to be features f_1^{obs} of size 64×1024 and f_2^{obs} of size 16×1024 .

We treat each featurized point in f_1^{demo} and f_1^{obs} as a token, concatenate them together (token-wise) and perform self attention among the concatenated tokens to get an updated feature $f_1^{obs'}$. The updated $f_1^{obs'}$ are then used for the next set abstraction layer, which generates f_2^{obs} . We then do the same for f_2^{demo} and f_2^{obs} , i.e., perform self attention among concatenated tokens of f_2^{demo} and f_2^{obs} to generate $f_2^{obs'}$. $f_2^{obs'}$ are then used for the subsequent feature propagation layers which will gradually up-sample the points. We further perform self attention between f_1^{demo} and the feature after the first feature propagation layer, to further incorporate the demonstration information (see Figure 1).

A.2 FiLM-based demonstration encoding

An illustration of this baseline is shown in Figure 2. Similar to the attention-based demonstration encoding baseline, in this baseline, to process the demonstration information, we first concatenate

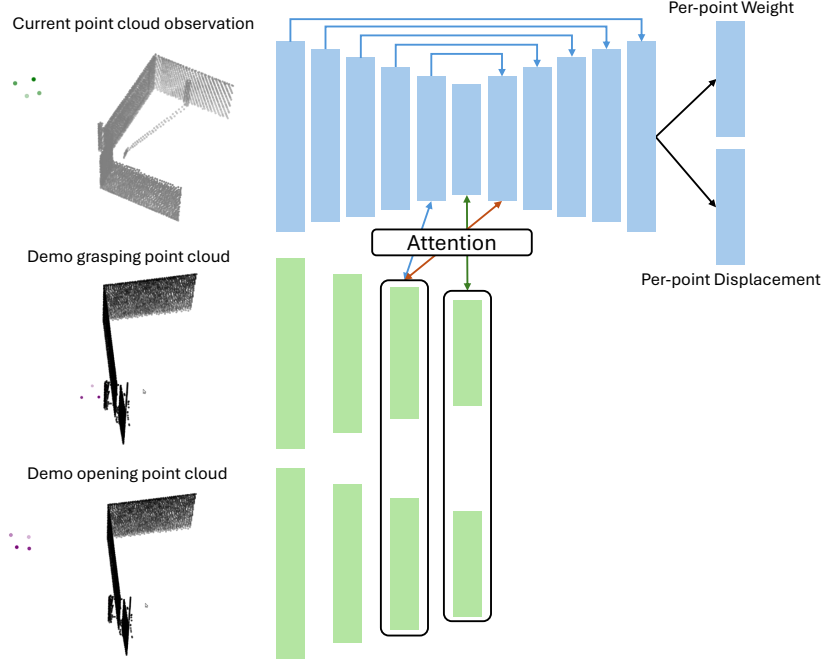


Figure 1: An illustration of the attention-based demonstration encoding baseline.

the demonstration object point cloud o_{demo} with the demonstration grasping end-effector points $\{ee_i^{demo-grasp}\}_{i=1}^4$ to obtain o_{demo}^{grasp} , and also concatenate the demonstration object point cloud o_{demo} with the demonstration opening end-effector points $\{ee_i^{demo-open}\}_{i=1}^4$ to obtain o_{demo}^{open} .

We then use a small PointNet++ to encode o_{demo}^{grasp} , o_{demo}^{open} , and the current observation o to obtain a latent embedding for each as f_{demo}^{grasp} , f_{demo}^{open} , and f^o . We then perform self attention between these three features and get an updated feature for f^o as f'^o .

As in ArticBot [2], we use another large PointNet++ to encode the current observation that predicts the weights and displacements for calculating the final predicted sub-goal end-effector points. It has 6 set abstraction and 6 feature propagation layers. The demonstration feature f'^o is encoded via FiLM into the feature generated after the 6th set abstraction layer, i.e., the bottleneck layer (See Figure 2).

A.3 Modification of our method on top of ArticBot [2]

We also make a modification to the loss when training the weighted displacement model in ArticBot: instead of using a L2 regression loss, we view the weighted displacement model as a Gaussian Mixture Model (GMM), where each point predicts a Gaussian distribution of the displacements, the weight for each point represents the weight for this Gaussian distribution, and all points' predictions form a Gaussian Mixture. We then train this GMM using negative log-likelihood loss. This GMM formulation should theoretically better handle multi-modal datasets, but in practice we do not find much performance difference compared to the original weighted displacement model, probably because our dataset does not contain much multi-modality.

B More simulation results

In table 1 in the main paper, we compared our method to a list of baselines. For both the Attention and FiLM-base ablation, similar to our method, we initialize the main PointNet++ architecture (blue layers in Figure 1 and Figure 2) that encodes the current observation point cloud to predict the

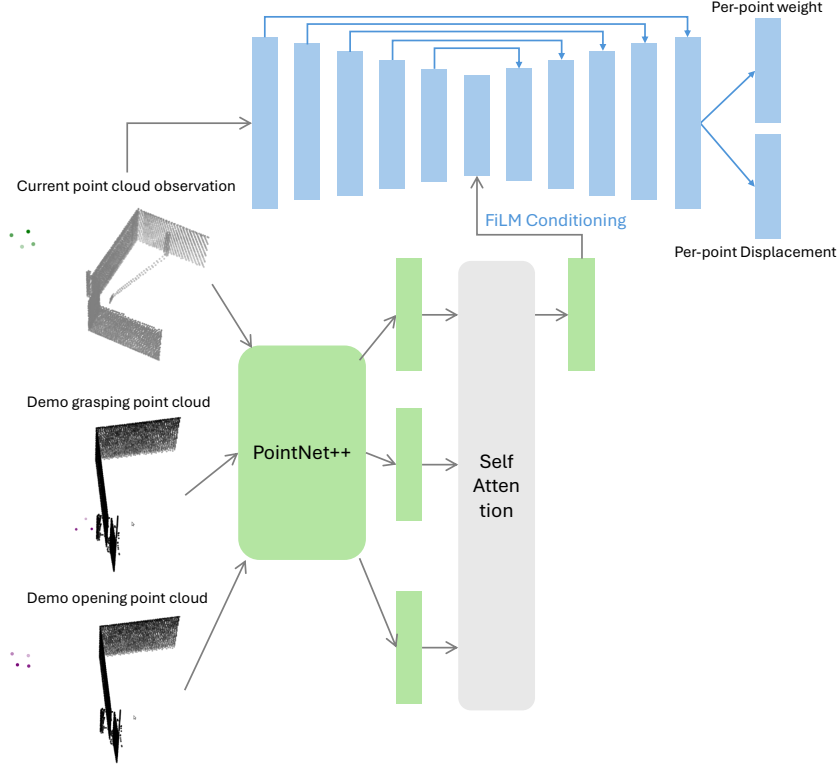


Figure 2: An illustration of the FiLM-based demonstration encoding baseline.

	ArticuBot	Ours	Attention	Attn w/ frozen pretrained model	FiLM	FiLM w/ frozen pretrained model
High-performing objects	0.778 $\pm$ 0.01	0.772 $\pm$ 0.01	0.557 $\pm$ 0.01	0.723 $\pm$ 0.01	0.630 $\pm$ 0.00	0.777 $\pm$ 0.01
Low-performing objects	0.579 $\pm$ 0.02	0.703 $\pm$ 0.02	0.209 $\pm$ 0.00	0.535 $\pm$ 0.01	0.371 $\pm$ 0.01	0.621 $\pm$ 0.00
All objects	0.703 $\pm$ 0.01	0.746 $\pm$ 0.01	0.427 $\pm$ 0.01	0.653 $\pm$ 0.01	0.533 $\pm$ 0.00	0.719 $\pm$ 0.00

Table 1: Normalized opening performance of all compared methods in simulation.

weights and displacements from a pre-trained policy that does not use any demonstration. For the FiLM-based demonstration encoding, we also compared to a baseline where we freeze the weights of the pre-trained main PointNet++ architecture (the blue layers in Figure 2), and find it to perform better compared to not freezing the weights.

Here we add a similar experiment for the attention-based demonstration encoding baseline, i.e., we freeze the weights of the pre-trained main PointNet++ architecture (the blue layers in Figure 1). The performance is shown in Table 1. As shown, freezing the weights lead to better performance compared to not freezing the weights for this attention-based demonstration encoding, but the performance is still worse compared to our proposed method.

C Visualization of Real World Results

See Figure 3 for a visualization of our method’s performance on 9 real-world test objects.

See Figure 4 for a comparison between ArticuBot’s policy rollouts and our method’s policy rollouts.

Figure 5 shows some of the failure cases of our method in the real world.

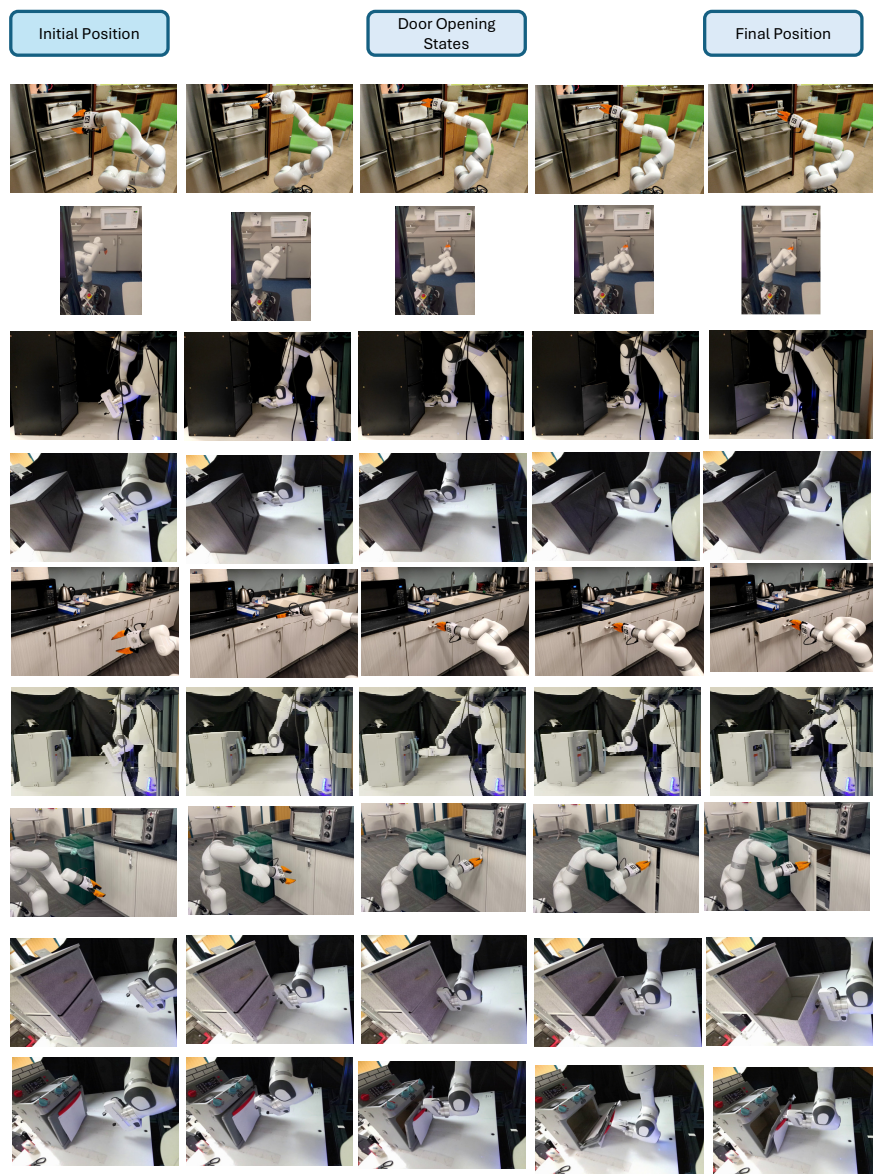


Figure 3: Screenshots of policy rollouts of our method on real world objects.

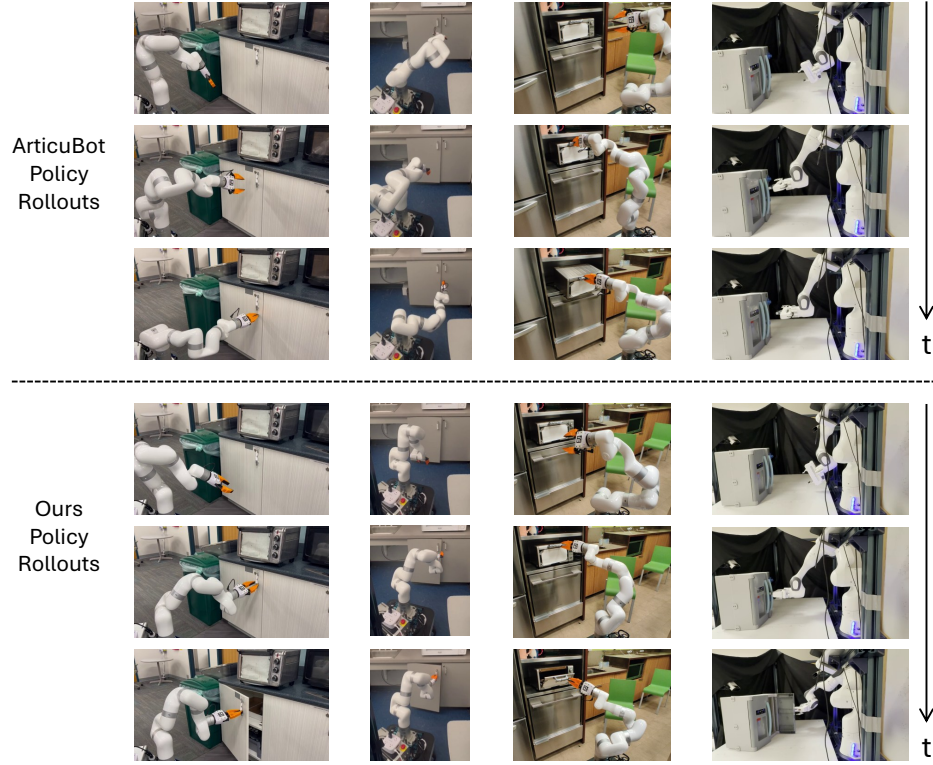


Figure 4: A comparison between ArticuBot’s policy rollouts and our method’s policy rollouts on 4 real world objects. Our policy successfully opens the object while ArticuBot fails.

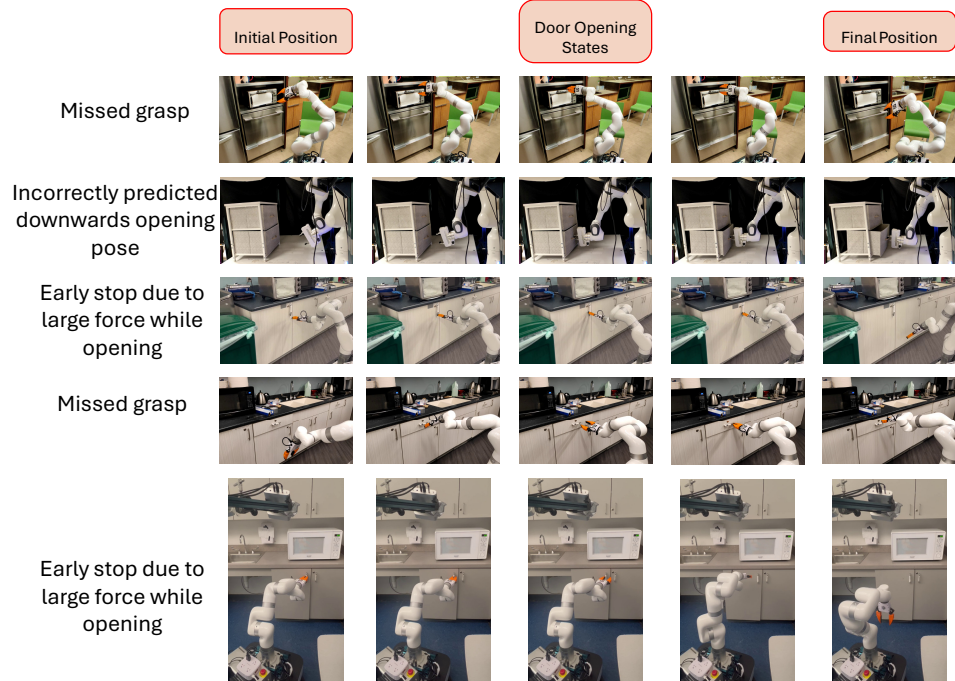


Figure 5: Some failure cases of our method.

83 **References**

- 84 [1] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. Pointnet++: Deep hierarchical feature learning on point
85 sets in a metric space. *Advances in neural information processing systems*, 30, 2017.
- 86 [2] Y. Wang, Z. Wang, M. Nakura, P. Bhowal, C.-L. Kuo, Y.-T. Chen, Z. Erickson, and D. Held.
87 Articubot: Learning universal articulated object manipulation policy via large scale simulation.
88 In *Robotics: Science and Systems (RSS)*, 2025.